Clase práctica — Realidad aumentada móvil en Unity 6

AR Mobile Template + AR Foundation + seguimiento de imágenes (Image Tracking) | Cátedra de Simulaciones — UAI | Actualizado: 25/10/2026

Propósito y resultados de aprendizaje

En esta clase partimos del AR Mobile Template oficial de Unity para comprender cómo una aplicación de realidad aumentada integra cámara, seguimiento del dispositivo, reconocimiento del entorno e interacción táctil. Primero exploramos la escena de ejemplo sin reprogramar lo que el template ya resuelve; después construimos una variante de reconocimiento de imágenes que muestra un objeto 3D cuando la cámara detecta un target impreso.

Al finalizar, cada estudiante podrá distinguir Unity XR, AR Foundation y los proveedores ARCore/ARKit; identificar AR Session, XR Origin y los managers; ejecutar la aplicación en un dispositivo Android; crear una Reference Image Library; y reaccionar al alta, actualización y pérdida de seguimiento de un target.

Plan sugerido: 120 minutos

0–15 min: explicación de AR frente a VR/MR y arquitectura del proyecto. 15–35 min: abrir el template y recorrer SampleScene. 35–55 min: configurar Android y ejecutar el ejemplo real. 55–90 min: preparar un target y construir Image Tracking. 90–110 min: probar estados de seguimiento y depurar. 110–120 min: conclusiones y presentación del trabajo práctico.

1. Qué es el AR Mobile Template y qué incluye hoy

El AR Mobile Template es una plantilla de proyecto, no un paquete que deba añadirse a una escena ya creada para convertirla automáticamente en un proyecto AR. Se selecciona al crear un proyecto desde Unity Hub. La documentación de la familia 2.1 identifica la versión 2.1.3 y describe una escena SampleScene en Assets/Scenes. Según la versión del Editor pueden aparecer versiones diferentes del template; verificamos la efectivamente instalada, en vez de dar por hecho que todas las versiones de Unity 6 traen el mismo contenido.

El template configura URP y los paquetes de AR Foundation, XR Interaction Toolkit, ARCore (Android) y ARKit (iOS). La escena ya contiene un XR Origin, AR Session, cámara con AR Camera Manager y AR Camera Background, AR Plane Manager, AR Raycast Manager, sistemas táctiles y UI. También ofrece un Object Spawner, un menú de creación y controles para seleccionar, mover o eliminar objetos. En clase conviene recorrer esta escena antes de

escribir código: los estudiantes pueden identificar qué resuelve la infraestructura y qué es comportamiento específico del ejemplo.
Ruta recomendada: Unity Hub → New Project → AR Mobile → elegir una versión del Editor compatible. Abrir Assets/Scenes/SampleScene.unity. No duplicar AR Session ni XR Origin: ya existen en el template.

Arquitectura: separar capas

AR Foundation es la API común para programación AR: expone managers como ARPlaneManager, ARRaycastManager y ARTrackedImageManager. ARCore XR Plug-in y ARKit XR Plug-in son implementaciones específicas para Android y iOS. XR Plug-in Management carga el proveedor apropiado. XR Interaction Toolkit organiza interacciones y entrada de usuario. El Input System traduce acciones táctiles, mientras que XR Origin relaciona la cámara y los elementos detectados con las coordenadas del mundo de Unity.
Flujo conceptual de la experiencia: cámara y sensores → ARCore/ARKit → AR Foundation → managers y trackables → lógica de la aplicación → objetos 3D / interfaz. La cámara del usuario no se mueve con Transform manualmente: su pose viene del subsistema XR.

2. Requisitos y configuración para Android

Equipo: Unity 6 con Android Build Support, Android SDK & NDK Tools y OpenJDK instalados desde Unity Hub; cable USB; teléfono compatible con ARCore; imagen objetivo impresa o presentada en otra pantalla. Para esta práctica se recomienda Android real: la simulación en Editor facilita probar recorridos e interfaz, pero no valida la detección real de una imagen impresa.
Ejemplo de aula: Samsung Galaxy A34 5G figura en el listado oficial de dispositivos compatibles con ARCore. En el teléfono activar Ajustes → Acerca del teléfono → Información de software → tocar siete veces Número de compilación; luego Opciones de desarrollador → Depuración USB y autorizar la conexión con la computadora.
En Unity: File → Build Profiles (o Build Settings, según versión del Editor) → Android → Switch Platform. En Project Settings → XR Plug-in Management → Android, confirmar que ARCore esté activado. En Player → Other Settings, usar IL2CPP y arquitectura ARM64. Para la práctica conviene OpenGLES3 si hay problemas de compatibilidad con la cámara; consultar la configuración sugerida por la versión instalada de ARCore.

Aclaración importante sobre la versión mínima de Android

La documentación actual de Google para aplicaciones AR Required con AR Foundation establece como mínimo Android 7.0 / API 24. No corresponde afirmar que ARCore exige universalmente API 26 (Android 8.0): esa cifra aparece en materiales antiguos y en algunas

configuraciones de ejemplo. Una versión de Unity, template o paquete puede imponer un mínimo superior. Para la clase, priorizar el mínimo que valide el Package Manager / Project Validation del proyecto instalado, y no confundir Minimum API Level con Target API Level, que además depende de requisitos de publicación.
Configurar ARCore Requirement como Required si la aplicación depende por completo de RA. Para una aplicación AR Optional existen comprobaciones adicionales de soporte e instalación. Verificar que Google Play Services for AR esté instalado o actualizado; ARSession.state sirve para observar el estado de la sesión.

Aviso de Active Input Handling = Both

Si Android muestra 'Unsupported Input Handling' por tener Active Input Handling = Both, no lo ignoramos como solución permanente. En Player → Other Settings → Active Input Handling seleccionamos Input System Package (New) para el template moderno. Atención: si algún asset externo emplea Input.GetAxis, Input.GetKeyDown o el sistema clásico, hay que migrar/adaptar esos scripts. Elegir solo Input Manager (Old) puede romper las interacciones del template basadas en Input System. La advertencia no significa que Android prohíba usar ambos sistemas en absolutamente todos los contextos; advierte de una configuración no soportada para ese modo de despliegue.

3. Recorrido guiado por la escena de ejemplo

Abrimos SampleScene y ubicamos: (a) AR Session, responsable del ciclo de vida; (b) XR Origin, origen de referencia; (c) Main Camera con AR Camera Manager + AR Camera Background; (d) AR Plane Manager, que detecta superficies; (e) AR Raycast Manager, que permite consultar una posición del mundo a partir de un punto de pantalla; (f) Object Spawner y el sistema táctil del ejemplo; (g) Canvas y mensajes de ayuda. Identificamos cuál de estos elementos pertenece a la infraestructura AR y cuál a la aplicación de demostración.
Actividad de observación: ejecutar la escena en Android, mover el teléfono lentamente alrededor de una mesa iluminada, detectar un plano, crear un objeto y desplazarlo.
Preguntas guía: ¿qué datos vienen de los sensores?, ¿qué pasa si ocultamos temporalmente la cámara?, ¿qué relación hay entre el toque en pantalla y la posición 3D del objeto?

Prueba en Editor con XR Simulation

El manual del template 2.1 permite habilitar XR Simulation como Desktop Plug-in Provider desde Project Settings → XR Plug-in Management e inicializar XR al comenzar. En Game View se puede recorrer un entorno con mouse y WASD, más Q/E para altura. Es un recurso

de enseñanza y pruebas rápidas, no un reemplazo de la cámara, el tracking o el rendimiento real del teléfono. Si la UI de la simulación no se visualiza correctamente, la propia guía del template sugiere configurar Android o iOS como build target.

4. Práctica principal: Image Tracking con un target

El objetivo ahora cambia: no buscamos un plano ni colocamos el objeto mediante tap. Queremos reconocer una imagen 2D específica —por ejemplo, una tarjeta, afiche, tapa de libro o marcador impreso— y mostrar contenido anclado a su pose. AR Foundation crea un ARTrackedImage cuando encuentra una imagen registrada en una librería de referencia. Debemos distinguir la detección inicial (added), los cambios de pose/estado (updated) y la eliminación del trackable (removed).

Paso A — Crear una escena de trabajo

Duplicar SampleScene y renombrar la copia ImageTargetLab. Conservar AR Session, XR Origin y su cámara configurada; desactivar o retirar el Object Spawner, las interacciones de colocar objetos y los menús propios del ejemplo para que la nueva práctica no mezcle dos comportamientos. En XR Origin agregar AR Tracked Image Manager, si aún no existe. No agregar otro XR Origin ni otra cámara AR por duplicado.

Paso B — Preparar el target físico

Importar a Assets/AR/Images un PNG o JPG de alto contraste, con suficientes detalles únicos, esquinas y texturas. Evitar imágenes casi lisas, patrones repetitivos y reflejos. Imprimirla, por ejemplo, a 15 × 15 cm; anotar el tamaño real. Cuando se prueba con otra pantalla se pueden introducir reflejos, escalado y diferencias de color: para evaluar, preferir impresión mate e iluminación uniforme.

Paso C — Crear la librería

Seleccionar Assets → Create → XR → Reference Image Library. Llamarla TargetsLibrary. Abrirla en el Inspector, usar Add Image, asociar la textura, asignar un nombre estable (por ejemplo, TarjetaLaboratorio) y activar Specify Size para ingresar el tamaño físico en metros: 0.15 m de ancho para una impresión de 15 cm. Si cambia el tamaño de impresión, debemos actualizar esa medida. En XR Origin → AR Tracked Image Manager, arrastrar TargetsLibrary a Serialized Library. Configurar Max Number Of Moving Images = 1 como punto de partida para la prueba.

Si el dispositivo debe seguir varios targets, añadir varias imágenes a la misma librería. El nombre de cada referencia es útil para distinguirlas por código, pero no es el algoritmo de

detección. Keep Texture at Runtime solo es necesario si la aplicación necesita acceder a esa textura durante la ejecución.

Paso D — Crear un prefab de contenido

Crear un prefab ContentOnTarget con un objeto 3D pequeño y claramente visible, por ejemplo un cubo elevado sobre la imagen. Mantener su escala en metros; para una tarjeta de 15 cm, un cubo de unos 5 cm de lado es apropiado. Diseñarlo suponiendo que el centro del target es el origen local y el eje normal local apunta hacia arriba. No dejar una instancia fija del prefab en la escena: se creará cuando se detecte una imagen.

Paso E — Script compatible con AR Foundation 6.x

En AR Foundation 6, la API recomendada de notificaciones es trackablesChanged (UnityEvent), no el antiguo trackedImagesChanged. El código siguiente instancia una sola vez por trackable, conserva el contenido como hijo de su transform y lo oculta cuando el seguimiento deja de ser confiable. Al volver a Tracking, lo muestra de nuevo. Los ejemplos que usan ‘trackedImagesChanged += ...’ pertenecen a una API obsoleta en AR Foundation 6.

```
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

[RequireComponent(typeof(ARTrackedImageManager))]
public class ImageTargetContent : MonoBehaviour
{
    [SerializeField] private GameObject contentPrefab;

    private ARTrackedImageManager manager;
    private readonly Dictionary<TrackableId, GameObject> content = new();

    private void Awake()
    {
        manager = GetComponent<ARTrackedImageManager>();
    }

    private void OnEnable()
    {
        manager.trackablesChanged.AddListener(OnImagesChanged);
    }

    private void OnDisable()
    {
        manager.trackablesChanged.RemoveListener(OnImagesChanged);
    }

    private void OnImagesChanged(
        ARTrackablesChangedEventArgs<ARTrackedImage> args)
    {
        foreach (ARTrackedImage image in args.added)
            UpdateContent(image);
```

```
        foreach (ARTrackedImage image in args.updated)
            UpdateContent(image);

        foreach (var removed in args.removed)
        {
            if (content.TryGetValue(removed.Key, out var go))
            {
                Destroy(go);
                content.Remove(removed.Key);
            }
        }
    }

    private void UpdateContent(ARTrackedImage image)
    {
        if (!content.TryGetValue(image.trackableId, out var go))
        {
            go = Instantiate(contentPrefab, image.transform);
            go.transform.localPosition = Vector3.zero;
            go.transform.localRotation = Quaternion.identity;
            go.transform.localScale = Vector3.one;
            content.Add(image.trackableId, go);
        }

        go.SetActive(image.trackingState == TrackingState.Tracking);
    }
}
```

Agregar ImageTargetContent al mismo XR Origin que posee AR Tracked Image Manager, y asignar ContentOnTarget al campo Content Prefab del script. No hace falta completar el campo Tracked Image Prefab del manager para esta estrategia: el manager genera los trackables y nuestro script crea el contenido hijo de cada uno. Verificar que el prefab tenga su geometría correctamente posicionada respecto del origen; si parece atravesar la imagen, elevar el mesh dentro del prefab sin mover el trackable.

Qué sucede cuando se pierde el target

Tracking indica seguimiento activo; Limited expresa seguimiento degradado y None ausencia de seguimiento. Que una imagen abandone el campo de visión no implica necesariamente un evento removed: puede conservarse con estado Limited. Por eso no creamos y destruimos objetos en cada frame; mantenemos el objeto asociado y solo alternamos su visibilidad, destruyéndolo cuando el subsistema elimina de verdad ese trackable. Nunca llamar Destroy sobre el GameObject del propio ARTrackedImage, porque pertenece al manager.

5. Build, ejecución y comprobaciones

En Android seleccionar Build & Run con la escena ImageTargetLab incluida en la lista de escenas del build. Con el A34 conectado y autorizado para depuración USB, instalar la app, aceptar el permiso de cámara y revisar que se inicie la sesión AR. Mostrar el target frente a la cámara, hacerlo girar y trasladarlo lentamente. El contenido debe seguir su posición y

orientación. Tapar la imagen parcialmente y retirarla del cuadro para observar los estados de tracking.
Prueba de criterio: al detectar una imagen, aparece un solo objeto; al moverla, el objeto acompaña la pose; al perder Tracking, desaparece visualmente; al recuperar Tracking, vuelve a aparecer sin duplicados. Como extensión, agregar una segunda imagen con contenido diferente, usando referenceImage.name y un conjunto explícito de prefabs.

6. Diagnóstico de problemas frecuentes

Pantalla negra o sin cámara: revisar permiso, AR Camera Manager / AR Camera Background, AR Session, proveedor ARCore, presencia de Google Play Services for AR y errores de Android Logcat. No asumir inmediatamente que falla el target si la cámara AR todavía no funciona.
No detecta la imagen: comprobar que el target esté en TargetsLibrary, la librería asociada en Serialized Library, el tamaño físico sea correcto y la imagen tenga detalles suficientes. Probar una impresión más grande y mate. Revisar estado de ARSession y la lista de dispositivos compatibles.
La imagen se detecta pero el prefab no aparece: verificar Content Prefab en Inspector, que el script y el manager estén activos, tamaño y posición del mesh, estado Tracking y errores de Console. Si hay errores sobre trackablesChanged o tipos de eventos, revisar la versión real de AR Foundation y no mezclar tutoriales de 5.x con APIs de 6.x.
Objetos desplazados, invertidos o de escala incorrecta: verificar la medida física ingresada en metros, el pivot del contenido, su orientación local y que no se esté multiplicando el transform del target por una escala adicional. Si el problema aparece solo al mover rápido el teléfono, revisar condiciones de iluminación y tracking.
Error de input por Both: usar Input System (New) para el template; si se conservan assets antiguos con Input.GetAxis, migrar ese input o aislar el comportamiento. No cambiar indiscriminadamente a Old si dependemos de interacciones táctiles de XRI.

7. Evaluación y actividad para entregar

Consigna: realizar una experiencia AR que reconozca dos imágenes distintas y muestre un objeto 3D diferente sobre cada una. La UI debe presentar al menos un mensaje de ayuda antes de detectar el target y un estado cuando se pierde el tracking. No se evalúa la calidad artística del modelo, sino la correcta integración entre la librería de imágenes, el tracking y la escena.
Entrega: proyecto Unity organizado (escena propia, Reference Image Library y prefabs), un video corto de funcionamiento real sobre un dispositivo y un README que indique versión del Editor, versión de AR Foundation, dispositivo y dificultades encontradas. Para una

extensión opcional, permitir rotar o escalar el contenido mediante un gesto, o registrar cuántas veces se detectó cada target.
Criterios: configuración técnica y ejecución real 30%; reconocimiento y seguimiento de dos targets 35%; gestión adecuada de Tracking/Limited/None 20%; organización y explicación de las decisiones 15%.

8. Ideas centrales para cerrar la clase

El dispositivo observa el mundo real y estima una pose, pero nuestro código no procesa manualmente los píxeles: ARCore/ARKit y AR Foundation exponen esa información como trackables. La diferencia entre plane detection e image tracking es el origen de la referencia espacial: una superficie genérica o una imagen concreta previamente registrada. En ambos casos, la aplicación debe distinguir resultados del subsistema y presentación visual para evitar duplicados y errores cuando cambia la calidad del seguimiento.

Documentación y referencias verificadas

AR Mobile Template 2.1.3 (Unity):
https://docs.unity3d.com/Packages/com.unity.template.ar-mobile@2.1/manual/index.html
AR Foundation 6.6.2 — Image Tracking:
https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@6.6/manual/features/image-tracking.html
AR Foundation 6.6.2 — AR Tracked Image Manager:
https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@6.6/manual/features/image-tracking/artrackedimagemanager.html
Reference Image Library:
https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@6.6/manual/features/image-tracking/reference-images.html
ARCore / AR Foundation — configuración Android:
https://developers.google.com/ar/develop/unity-arf/enable-arcore
ARCore — dispositivos compatibles: https://developers.google.com/ar/devices
Ejemplos oficiales de Unity: https://github.com/Unity-Technologies/arfoundation-samples
Nota de actualización: contrastado con documentación oficial al 25/10/2026. Verificar siempre el número exacto de paquetes instalado en Package Manager, especialmente si el laboratorio mezcla distintas versiones de Unity 6.